

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 5
«ПРОЦЕДУРЫ, ФУНКЦИИ, ТРИГГЕРЫ В PostgreSQL» по
дисциплине «Проектирование и реализация баз данных»

Обучающийся Лукошников Дмитрий Викторович

Факультет прикладной информатики

Группа K3242

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии

Преподаватель Говорова Марина Михайловна

Санкт-Петербург

2026

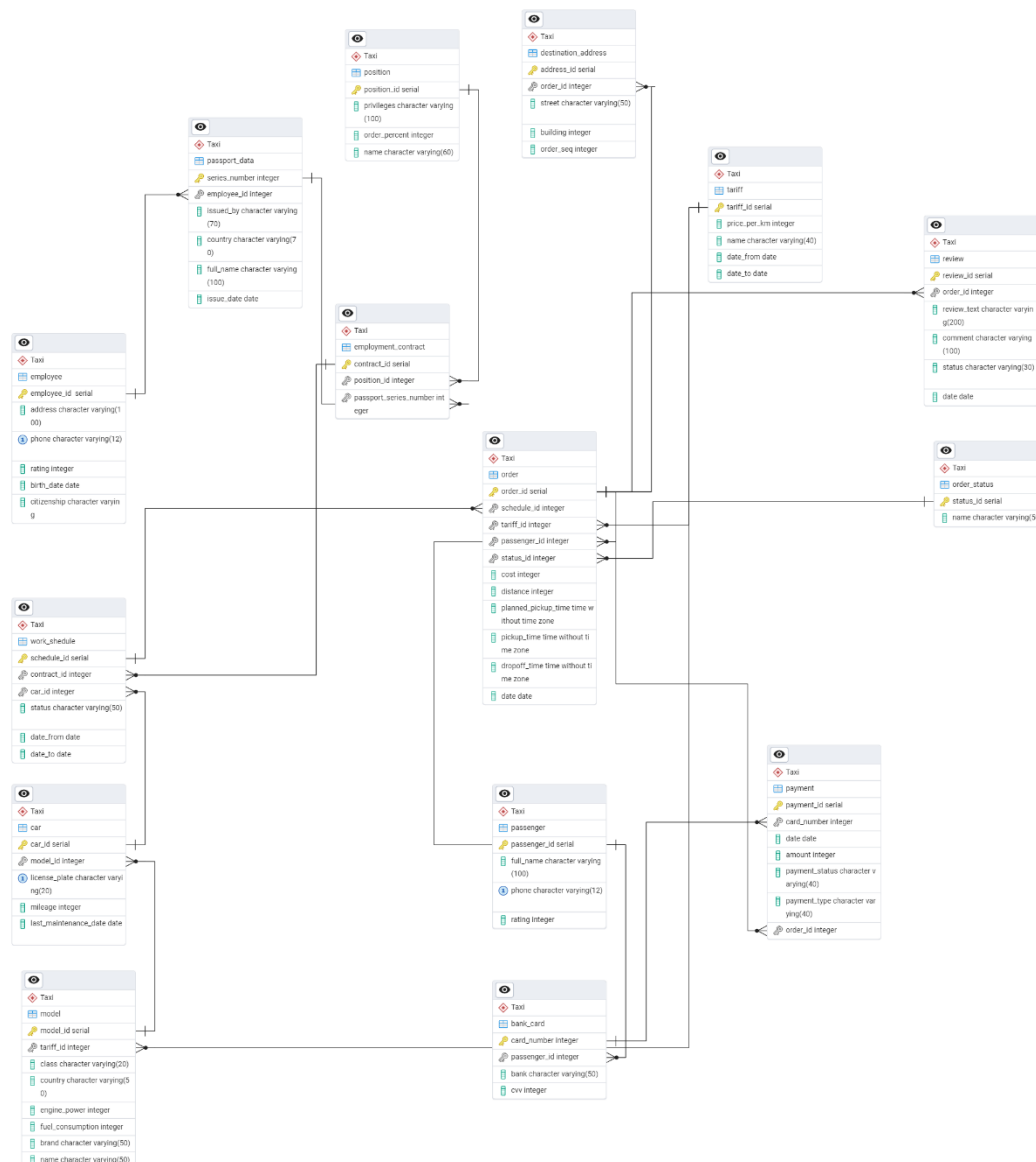
Цель работы:

Овладеть практическими навыками создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

Практическое задание:

1. Создать 3 процедуры для индивидуальной БД согласно варианту (часть 4 ЛР 2). Допустимо использование IN/INOUT параметров. Допустимо создать авторские процедуры.
2. Создать триггеры для индивидуальной БД согласно варианту. Вариант 2.1 — 7 триггеров по ИЗ.

Схема базы данных (ЛР 3).



Выполнение:

Процедуры

Процедура 1. Для вывода данных о пассажирах, которые заказывали такси в заданном, как параметр, временном интервале.

```
CREATE OR REPLACE PROCEDURE passenger_data_by_date(start_date DATE, end_date DATE)
LANGUAGE plpgsql
AS $$
DECLARE
    rec RECORD;
BEGIN
    FOR rec IN
        SELECT
            p.passenger_id,
            p.full_name,
            p.phone,
            p.rating
        FROM "Taxi".passenger p
        JOIN "Taxi"."order" o ON p.passenger_id = o.passenger_id
        WHERE o.date BETWEEN start_date AND end_date
        GROUP BY p.passenger_id, p.full_name, p.phone, p.rating
    LOOP
        RAISE NOTICE 'ID: %, Name: %, Phone: %, Rating: %',
            rec.passenger_id,
            rec.full_name,
            rec.phone,
            rec.rating;
    END LOOP;
END;
$;$
```

```
CREATE PROCEDURE
TaxiOrder=#
TaxiOrder=# call passenger_data_by_date('2024-10-05', '2024-10-10');
ЗАМЕЧАНИЕ: ID: 1, Name: Иванов Иван Иванович, Phone: +79161112233, Rating: 4
ЗАМЕЧАНИЕ: ID: 2, Name: Петрова Мария Сергеевна, Phone: +79162223344, Rating: 5
ЗАМЕЧАНИЕ: ID: 3, Name: Сидоров Алексей Петрович, Phone: +79163334455, Rating: 3
ЗАМЕЧАНИЕ: ID: 4, Name: Козлова Елена Владимировна, Phone: +79164445566, Rating: 4
ЗАМЕЧАНИЕ: ID: 9, Name: Степанов Артем Игоревич, Phone: +79169990011, Rating: 4
ЗАМЕЧАНИЕ: ID: 10, Name: Дмитриева Наталья Павловна, Phone: +79160001122, Rating: 4
ЗАМЕЧАНИЕ: ID: 11, Name: Андреев Максим Романович, Phone: +79169991122, Rating: 4
ЗАМЕЧАНИЕ: ID: 12, Name: Романов Виктор Викторович, Phone: +79162227788, Rating: 5
```

вызов CALL

Процедура 2. Вывести сведения о том, куда был доставлен пассажир по заданному номеру телефона пассажира

```
CREATE OR REPLACE PROCEDURE destination_by_number2(phone_number VARCHAR(12))
LANGUAGE plpgsql
AS $$
DECLARE
```

```

        rec RECORD;
BEGIN
    RAISE NOTICE 'Адреса доставки для пассажира с номером %:', phone_number;
    RAISE NOTICE '-----';

    FOR rec IN
        SELECT
            p.full_name,
            da.street,
            da.building
        FROM "Taxi".passenger p
        JOIN "Taxi"."order" o ON p.passenger_id = o.passenger_id
        JOIN "Taxi".destination_address da ON o.order_id = da.order_id
        WHERE p.phone = phone_number
        ORDER BY o.date DESC
    LOOP
        RAISE NOTICE 'Пассажир: %, Улица: %, Дом: %',
            rec.full_name,
            rec.street,
            rec.building;
    END LOOP;
END;
$$;

```

```

TaxiOrder=# call destination_by_number2('+79161112233');
ЗАМЕЧАНИЕ: Адреса доставки для пассажира с номером +79161112233:
ЗАМЕЧАНИЕ: -----
ЗАМЕЧАНИЕ: Пассажир: Иванов Иван Иванович, Улица: ул. Тверская, Дом: 15
ЗАМЕЧАНИЕ: Пассажир: Иванов Иван Иванович, Улица: ул. Большая Дмитровка, Дом: 12

```

вызов CALL

Процедура 3. Для вычисления суммарного дохода таксопарка за истекший месяц.

```

CREATE OR REPLACE PROCEDURE total_sum_last_month2()
LANGUAGE plpgsql
AS $$
DECLARE
    last_month_start DATE;
    last_month_end DATE;
    total_income BIGINT;
    orders_count BIGINT;
BEGIN
    last_month_start := (DATE_TRUNC('month', CURRENT_DATE) - INTERVAL '1
month')::DATE;
    last_month_end := (DATE_TRUNC('month', CURRENT_DATE) - INTERVAL '1
day')::DATE;

    SELECT
        COALESCE(SUM(o.cost), 0),
        COUNT(o.order_id)
    INTO total_income, orders_count
    FROM "Taxi"."order" o

```

```

WHERE o.date BETWEEN last_month_start AND last_month_end;

RAISE NOTICE '-----';
RAISE NOTICE 'ОТЧЕТ О ДОХОДАХ ТАКСОПАРКА';
RAISE NOTICE '-----';
RAISE NOTICE 'Период: % - %', last_month_start, last_month_end;
RAISE NOTICE '-----';
RAISE NOTICE 'Суммарный доход: % руб.', total_income;
RAISE NOTICE 'Количество заказов: %', orders_count;
RAISE NOTICE '-----';
END;
$$;

```

```
TaxiOrder=# call total_sum_last_month2();
```

```

ЗАМЕЧАНИЕ: -----
ЗАМЕЧАНИЕ: ОТЧЕТ О ДОХОДАХ ТАКСОПАРКА
ЗАМЕЧАНИЕ: -----
ЗАМЕЧАНИЕ: Период: 2026-04-01 - 2026-04-30
ЗАМЕЧАНИЕ: -----
ЗАМЕЧАНИЕ: Суммарный доход: 5000300 руб.
ЗАМЕЧАНИЕ: Количество заказов: 2
ЗАМЕЧАНИЕ: -----

```

Вызов call

Триггеры

Триггер 1. Авто формат телефона

(Телефон должен иметь начальный код '+7' и следующие 10 цифр)

```

CREATE OR REPLACE FUNCTION fmt_phone()
RETURNS TRIGGER AS $$
DECLARE
    digits_only TEXT;
BEGIN
    digits_only := regexp_replace(NEW.phone, '^[0-9]', '', 'g');
    NEW.phone := '+7' || RIGHT(digits_only, 10);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_fmt_phone
BEFORE INSERT OR UPDATE ON "Taxi".passenger
FOR EACH ROW EXECUTE FUNCTION fmt_phone();

```

```

CREATE TRIGGER
TaxiOrder=# INSERT INTO "Taxi".passenger (full_name, phone, rating) VALUES
TaxiOrder=# ('Тест1_Россия_8', '89261234567', 5),
TaxiOrder=# ('Тест2_Плюс7', '+79534234567', 5),
TaxiOrder=# ('Тест3_С_дефисами', '8-926-999-45-67', 5),
TaxiOrder=# ('Тест4_Со_скобками', '8 (926) 123-99-67', 5);
INSERT 0 4
TaxiOrder=# SELECT * FROM "Taxi".passenger;

```

passenger_id	full_name	phone	rating
1	Иванов Иван Иванович	+79161112233	4
2	Петрова Мария Сергеевна	+79162223344	5
3	Сидоров Алексей Петрович	+79163334455	3
4	Козлова Елена Владимировна	+79164445566	4
5	Михайлов Дмитрий Андреевич	+79165556677	5
6	Федорова Ольга Николаевна	+79166667788	4
7	Николаев Павел Сергеевич	+79167778899	3
8	Алексеева Анна Викторовна	+79168889900	5
9	Степанов Артем Игоревич	+79169990011	4
10	Дмитриева Наталья Павловна	+79160001122	4
11	Андреев Максим Романович	+79169991122	4
12	Романов Виктор Викторович	+79162227788	5
13	Новичков Николай Нетзаказов	+79990001122	5
21	52	+79866433333	4
26	Тест1_Россия_8	+79261234567	5
27	Тест2_Плюс7	+79534234567	5
28	Тест3_С_дефисами	+79269994567	5
29	Тест4_Со_скобками	+79261239967	5

(18 строк)

Проверка правильности работы триггера при разных вариантах телефона

Триггер 2. Автоматически рассчитывает стоимость заказа

(Берет стоимость за км у данного тарифа и умножает на дистанцию)

```

CREATE OR REPLACE FUNCTION calc_cost()
RETURNS TRIGGER AS $$
DECLARE
    price INTEGER;
BEGIN
    SELECT price_per_km INTO price FROM "Taxi".tariff WHERE tariff_id =
NEW.tariff_id;
    NEW.cost := NEW.distance * price;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_calc_cost
BEFORE INSERT ON "Taxi"."order"
FOR EACH ROW EXECUTE FUNCTION calc_cost();

```

```
TaxiOrder=# INSERT INTO "Taxi"."order" (schedule_id, tariff_id, passenger_id, status_id, distance, planned_pickup_time, date)
TaxiOrder=# VALUES (1, 1, 1, 1, 10, '12:00:00', CURRENT_DATE);
INSERT 0 1
TaxiOrder=# SELECT cost, order_id, date FROM "Taxi"."order" order by order_id;
```

cost	order_id	date
450	1	2026-03-02
780	2	2026-03-02
320	3	2026-03-02
560	4	2026-03-02
890	5	2026-03-02
430	6	2024-10-03
1240	7	2024-10-04
670	8	2024-10-04
350	9	2024-10-05
520	10	2024-10-05
980	11	2024-10-06
430	12	2024-10-06
670	13	2024-10-07
890	14	2024-10-07
500	15	2024-10-10
600	16	2024-10-10
450	17	2024-10-11
550	18	2024-10-11
300	19	2026-04-23
5000000	20	2026-04-23
5360	22	2026-03-02
5360	24	2026-03-02
5360	25	2026-03-02
6432	26	2026-03-02
5360	27	2026-03-02
300	28	2026-05-08
700	29	2026-03-02
6432	30	2026-03-02
300	31	2026-05-22

(29 строк)

Проверка работа триггера

Видно, что в последней строке, которую мы вставили автоматически рассчиталась стоимость

Триггер 3. Запрещает удаление водителя, если у него есть активные (незавершённые) заказы

```
CREATE OR REPLACE FUNCTION no_delete_driver()
RETURNS TRIGGER AS $$
DECLARE
    cnt INTEGER;
BEGIN
    SELECT COUNT(*) INTO cnt FROM "Taxi"."order" o
    JOIN "Taxi".work_shedule ws ON o.schedule_id = ws.schedule_id
    JOIN "Taxi".employment_contract ec ON ws.contract_id = ec.contract_id
    JOIN "Taxi".passport_data pd ON pd.series_number =
ec.passport_series_number
    WHERE pd.employee_id = OLD.employee_id AND o.status_id != 3;

    IF cnt > 0 THEN RAISE EXCEPTION 'У водителя есть активные заказы (%)',
cnt;
    END IF;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_no_delete_driver
BEFORE DELETE ON "Taxi".employee
FOR EACH ROW EXECUTE FUNCTION no_delete_driver();
```

```

CREATE TRIGGER
TaxiOrder=# select e.employee_id, pd.full_name, o.status_id from "Taxi".employee e
TaxiOrder=# JOIN "Taxi".passport_data pd ON pd.employee_id = e.employee_id
TaxiOrder=# JOIN "Taxi".employment_contract ec ON pd.series_number = ec.passport_series_number
TaxiOrder=# JOIN "Taxi".work_schedule ws ON ws.contract_id = ec.contract_id
TaxiOrder=# JOIN "Taxi".order o ON o.schedule_id = ws.schedule_id
TaxiOrder=# WHERE o.status_id != 3;

```

employee_id	full_name	status_id
1	Сергеев Сергей Петрович	1
1	Сергеев Сергей Петрович	1
1	Сергеев Сергей Петрович	1
1	Сергеев Сергей Петрович	1
5	Кузнецов Андрей Сергеевич	2
5	Кузнецов Андрей Сергеевич	2
5	Кузнецов Андрей Сергеевич	2
5	Кузнецов Андрей Сергеевич	2
5	Кузнецов Андрей Сергеевич	2
5	Кузнецов Андрей Сергеевич	1

(10 строк)

```

TaxiOrder=# delete from "Taxi".employee where employee_id = 5;
ОШИБКА: У водителя есть активные заказы (6)
КОНТЕКСТ: функция PL/pgSQL no_delete_driver(), строка 11, оператор RAISE

```

Проверка работы триггера

Мы видим, что у Кузнецова Андрея есть 6 незавершенных заказов(status_id=2 – в процессе, status_id=1 - не начат) и нам не дает удалить его из БД

Триггер 4. Контроль количества символов в отзыве

(Нельзя добавить отзыв, у которого текст меньше 10 или больше 500)

```

CREATE OR REPLACE FUNCTION check_review_length()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.review_text IS NOT NULL AND LENGTH(NEW.review_text) < 10 THEN
        RAISE EXCEPTION 'Отзыв слишком короткий (минимум 10 символов, введено %)',
        LENGTH(NEW.review_text);
    END IF;

    IF NEW.review_text IS NOT NULL AND LENGTH(NEW.review_text) > 500 THEN
        RAISE EXCEPTION 'Отзыв слишком длинный (максимум 500 символов, введено
    %)', LENGTH(NEW.review_text);
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_review_length
BEFORE INSERT OR UPDATE ON "Taxi".review
FOR EACH ROW
EXECUTE FUNCTION check_review_length();

```

```

TaxiOrder=# INSERT INTO "Taxi".review (order_id, review_text,comment, status, date)
TaxiOrder=# values (2,'Мало', 'Круто', 'опубликован', '2024-10-01');
ОШИБКА: Отзыв слишком короткий (минимум 10 символов, введено 4)
КОНТЕКСТ: функция PL/pgSQL check_review_length(), строка 4, оператор RAISE

```

Работа триггера

Не дает добавить короткий отзыв

Триггер 5. Автоматическая установка статуса закончен при заполнении времени высадки

(Если есть время высадки, значит заказ завершен)

```
CREATE OR REPLACE FUNCTION auto_complete_order()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.dropoff_time IS NOT NULL AND NEW.status_id != 3 THEN
        NEW.status_id := (SELECT status_id FROM "Taxi".order_status WHERE name =
            'закончен');
        RAISE NOTICE 'Заказ % автоматически завершён (время высадки: %)',
            NEW.order_id, NEW.dropoff_time;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER trg_auto_complete_order
BEFORE INSERT OR UPDATE ON "Taxi"."order"
FOR EACH ROW
EXECUTE FUNCTION auto_complete_order();
```

```
TaxiOrder=# INSERT INTO "Taxi"."order" (schedule_id,tariff_id,passenger_id,status_id,cost,distance,planned_pickup_time,pickup_time,dropoff_time,date)
TaxiOrder=# values (5,3,5,2,700,67,'09:15:00','09:18:00','09:55:00','2026-03-02');
ЗАМЕЧАНИЕ: Заказ 32 автоматически завершён (время высадки: 09:55:00)
INSERT 0 1
```

```
TaxiOrder=# select order_id, status_id from "Taxi".order where
order_id | status_id
-----+-----
          32 |          3
(1 строка)
```

Работа триггера

Видим, что при вставке новой строки с заполненным временем высадки, триггер срабатывает и ставит статус заказа(3) = завершен

Триггер 6. Запрет отзыва без завершённого заказа

(Нельзя оставлять отзыв, пока заказ не завершится)

```
CREATE OR REPLACE FUNCTION check_review_allowed()
RETURNS TRIGGER AS $$
DECLARE
    order_status_id INTEGER;
BEGIN
    SELECT status_id INTO order_status_id
    FROM "Taxi"."order"
    WHERE order_id = NEW.order_id;

    IF order_status_id != (SELECT status_id FROM "Taxi".order_status WHERE name =
        'закончен') THEN
```

```

        RAISE EXCEPTION 'Нельзя оставить отзыв на незавершённый заказ (статус не
"закончен")';
    END IF;

```

```

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER trg_check_review_allowed
    BEFORE INSERT ON "Taxi".review
    FOR EACH ROW
    EXECUTE FUNCTION check_review_allowed();

```

```

TaxiOrder=# INSERT INTO "Taxi".review (order_id, review_text,comment, status, date)
TaxiOrder-# values (5,'Не должно быть', 'Круто', 'опубликован', '2024-10-01');
ОШИБКА: Нельзя оставить отзыв на незавершённый заказ (статус не "закончен")
КОНТЕКСТ: функция PL/pgSQL check_review_allowed(), строка 10, оператор RAISE

```

Работа триггера

Нельзя оставить отзыв на незавершенный заказ

Триггер 7. Автоматически применяет ночной коэффициент к стоимости заказа, если поездка совершается после 22:00 и до 7:00

```

CREATE OR REPLACE FUNCTION apply_night_rate()
RETURNS TRIGGER AS $$
DECLARE
    base_price INTEGER;
    night_rate NUMERIC := 1.2;
BEGIN
    SELECT price_per_km INTO base_price
    FROM "Taxi".tariff
    WHERE tariff_id = NEW.tariff_id;

    NEW.cost := NEW.distance * base_price;

    IF NEW.planned_pickup_time >= '22:00:00' OR NEW.planned_pickup_time <
'07:00:00' THEN
        NEW.cost := NEW.cost * night_rate;
        RAISE NOTICE 'Применён ночной коэффициент %. Стоимость: % руб.',
night_rate, NEW.cost;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE OR REPLACE TRIGGER trg_apply_night_rate
    BEFORE INSERT ON "Taxi"."order"
    FOR EACH ROW
    EXECUTE FUNCTION apply_night_rate();

```

```
TaxiOrder=# INSERT INTO "Taxi"."order" (schedule_id,tariff_id,passenger_id,status_id,cost,distance,planned_pickup_time,pickup_time,dropoff_time,date)
TaxiOrder=# values (5,3,5,2,700,67,'23:00:00','23:00:00',NULL,'2026-03-02');
ЗАМЕЧАНИЕ: Применён ночной коэффициент 1.2. Стоимость: 6432 руб.
INSERT 0 1
```

```
TaxiOrder=# select order_id, cost, planned_pickup_time from "Taxi".order
order_id | cost | planned_pickup_time
-----+-----+-----
      33 | 6432 | 23:00:00
(1 строка)
```

Работа триггера

Видим, что на ночной заказ применен соответствующий коэффициент

Вывод:

В ходе выполнения лабораторной работы №5 были успешно освоены практические навыки создания и использования хранимых процедур и триггеров в СУБД PostgreSQL на примере предметной области «Таксопарк». Разработаны три соответствующие процедуры для вывода пассажиров за заданный период, получения адресов доставки по номеру телефона и расчёта суммарного дохода за истекший месяц, что позволило закрепить понимание различий между функциями и процедурами. Создано семь оригинальных триггеров, автоматизирующих ключевые бизнес-процессы: форматирование телефонов, расчёт стоимости заказа, запрет удаления водителя с активными заказами, контроль длины отзыва, автоматическое завершение заказа при указании времени высадки, запрет отзыва на незавершённый заказ и применение ночного коэффициента к стоимости поездки. Все триггеры были протестированы на корректность, подтверждено их срабатывание в различных ситуациях, включая попытки нарушения ограничений. Использование хранимых процедур позволяет централизовать бизнес-логику и упростить клиентские приложения, а триггеры обеспечивают автоматическую поддержку ссылочной целостности и контроль корректности данных на уровне сервера. Полученные навыки являются важной частью администрирования и разработки баз данных, позволяя повысить надёжность и безопасность информационных систем. Вся работа выполнена в консоли psql с фиксацией результатов в виде скриншотов для отчёта.